EXPERIMENT 9: APACHE SPARK

Single-machine Ubuntu practical guide — commands and what each step does

Aim: Use Apache Spark for data processing with RDDs, DataFrames, and SQL.

How to use this guide: Run each command in order. Shell commands go in Ubuntu Terminal. Scala commands go only after the Spark shell displays the **scala>** prompt.

Important: The commands below use Spark 4.0.1 with Java 17 as an example. If Spark/Java are already installed, check their versions first rather than reinstalling. Use the same folder names shown below or adjust paths to match your installation.

STEP 1 — UPDATE UBUNTU

What this does: Refreshes Ubuntu's package list.

```
sudo apt update
```

STEP 2 — INSTALL JAVA, SCALA, PYTHON, AND WGET

What this does: Installs Java for Spark, Scala for spark-shell, Python for PySpark, pip, and wget.

```
sudo apt install openjdk-17-jdk scala python3 python3-pip wget -y
java -version
scala -version
python3 --version
```

STEP 3 — DOWNLOAD SPARK

What this does: Downloads the Spark 4.0.1 archive with Hadoop 3 support.

```
cd ~/Downloads
wget https://archive.apache.org/dist/spark/spark-4.0.1/spark-4.0.1-bin-hadoop3.tgz
```

STEP 4 — EXTRACT AND MOVE SPARK

What this does: Unpacks Spark and moves it to /usr/local/spark. Skip the move if that destination already exists; check it first.

```
tar -xzf spark-4.0.1-bin-hadoop3.tgz
sudo mv spark-4.0.1-bin-hadoop3 /usr/local/spark
sudo chown -R "$USER":"$USER" /usr/local/spark
```

STEP 5 — CONFIGURE ENVIRONMENT VARIABLES

What this does: Sets the Java and Spark paths for your terminal. Check that the Java 17 directory exists with ls /usr/lib/jvm/ first.

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc
echo 'export SPARK_HOME=/usr/local/spark' >> ~/.bashrc
echo 'export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin' >> ~/.bashrc
source ~/.bashrc
echo "$JAVA_HOME"
echo "$SPARK_HOME"
```

STEP 6 — CREATE INPUT FILES

What this does: Creates a text file for word count and a CSV file for DataFrame/SQL practice.

```
echo "Spark is fast and Spark is powerful" > ~/input.txt
echo "id,name,age" > ~/students.csv
echo "1,Alice,22" >> ~/students.csv
echo "2,Bob,25" >> ~/students.csv
```

STEP 7 — START SPARK SHELL

What this does: Opens Spark's interactive Scala environment. Run the following Scala steps only after the scala> prompt appears.

```
spark-shell
```

STEP 8 — READ THE TEXT FILE

What this does: Reads the file into an RDD. This is a Scala command inside spark-shell.

```
val inputFile = sc.textFile("file:///home/" + sys.env("USER") + "/input.txt")
```

STEP 9 — COUNT LINES

What this does: Counts the number of lines in the RDD.

```
inputFile.count()
```

STEP 10 — DISPLAY FILE CONTENT

What this does: Collects the RDD contents and prints each line.

```
inputFile.collect().foreach(println)
```

STEP 11 — SPLIT LINES INTO WORDS

What this does: Turns each line into individual words.

```
val words = inputFile.flatMap(line => line.split(" "))
```

STEP 12 — CREATE WORD-COUNT PAIRS

What this does: Maps every word to a key-value pair with an initial count of 1.

```
val pairs = words.map(word => (word, 1))
```

STEP 13 — ADD COUNTS FOR REPEATED WORDS

What this does: Combines values with the same word key.

```
val counts = pairs.reduceByKey(_ + _)
```

STEP 14 — DISPLAY WORD COUNTS

What this does: Prints the word and its total count. Output order can vary.

```
counts.collect().foreach(println)
```

Expected result (order can vary):

```
(Spark,2)
(is,1)
(fast,1)
(and,1)
(powerful,1)
```

STEP 15 — LOAD CSV INTO A DATAFRAME

What this does: Reads students.csv and treats its first row as column names.

```
val df = spark.read
  .option("header", "true")
  .csv("file:///home/" + sys.env("USER") + "/students.csv")
```

STEP 16 — DISPLAY THE DATAFRAME

What this does: Shows the student records in tabular form.

```
df.show()
```

Expected output:

```
+---+-----+---+
| id| name|age|
+---+-----+---+
|  1|Alice| 22|
|  2|  Bob| 25|
+---+-----+---+
```

STEP 17 — DISPLAY THE SCHEMA

What this does: Shows column names and their data types. CSV columns are strings by default unless schema inference is enabled.

```
df.printSchema()
```

Expected output:

```
root
 |-- id: string (nullable = true)
 |-- name: string (nullable = true)
 |-- age: string (nullable = true)
```

STEP 18 — CREATE A TEMPORARY SQL VIEW

What this does: Registers the DataFrame under the temporary view name students.

```
df.createOrReplaceTempView("students")
```

STEP 19 — DISPLAY ALL STUDENTS WITH SQL

What this does: Executes an SQL SELECT query and displays all rows.

```
spark.sql("SELECT * FROM students").show()
```

STEP 20 — COUNT STUDENT RECORDS

What this does: Counts rows in the students view.

```
spark.sql("SELECT COUNT(*) AS total FROM students").show()
```

STEP 21 — EXIT SPARK SHELL

What this does: Closes the interactive Scala shell and returns to Ubuntu Terminal.

```
:quit
```

RESULT

Spark reads a text file into an RDD, counts words, loads CSV data into a DataFrame, displays its schema, and executes SQL queries.

TROUBLESHOOTING

• If spark-shell is not found, check SPARK_HOME and PATH, then run source ~/.bashrc. • If a path does not exist, check the actual folder name under /usr/local and the Java directories under /usr/lib/jvm. • If Spark is already installed, do not overwrite it without checking its version first. • Spark standalone mode does not configure HDFS; HDFS integration requires a separate working Hadoop setup.